

Informe de Experimentos — Simulador de Scheduling

Fecha: 17 de octubre de 2025

Objetivo

Comparar objetivamente tres algoritmos de scheduling (FCFS, Round Robin, SJF) frente a tres tipos de workloads definidos por la proporción de procesos I/O-bound y CPU-bound: balanceado (50/50), I/O-bound dominante (90% I/O), y CPU-bound dominante (90% CPU). Las métricas evaluadas son:

- Throughput (procesos completados / unidad de tiempo)
- Avg Turnaround (promedio de tiempo turnaround)
- Avg Response Time (promedio de tiempo de respuesta)

Requerimientos (resumen)

Para el trabajo se definieron los siguientes requerimientos mínimos:

1. Implementar un simulador en C que modele procesos con bursts (CPU e I/O).
2. Soportar la selección de la política de scheduling en tiempo de ejecución: FCFS, Round Robin (configurable quantum), SJF.
3. Permitir generar/usar workloads con 100 procesos, y con 10 bursts por proceso (se definió esta estructura en el simulador).
4. Registrar trazas de eventos (llegada, entrada/salida de CPU, espera por I/O, finalización) y calcular métricas finales.
5. Ejecutar experiments para los tres escenarios de mezcla de procesos (50/50, 90% I/O, 90% CPU) y para cada algoritmo.

Implementación (breve)

El simulador modela: - Tipo de bursts: CPU_BURST, IO_BURST. - Estados de proceso: NEW, READY, RUNNING, WAITING, TERMINATED. - Estructuras de colas: cola FIFO para FCFS y RR, y cola por prioridad (duración del próximo CPU burst) para SJF. - Dispatch y actualización por tick de reloj: llegada de procesos, decremento de bursts, manejo de I/O y preempción para Round Robin.

Se añadieron generadores dinámicos de workloads que crean 100 procesos con una fracción de I/O especificada (por ejemplo 0.90 para 90% I/O-bound). Cada proceso obtiene 10 bursts generados aleatoriamente respetando la clasificación I/O/CPU-bound (probabilidades y distribuciones de duración configurables en el código).

Workloads utilizados

- Balanced (50% I/O, 50% CPU). 100 procesos.
- I/O-heavy (90% I/O, 10% CPU). 100 procesos.
- CPU-heavy (10% I/O, 90% CPU). 100 procesos.

Los procesos se hicieron llegar en el tiempo agrupando 10 procesos por unidad de tiempo ($\text{arrival_time} = \text{index} / 10$), para distribuir arriivos durante 10 unidades de tiempo.

Resultados medidos

Los resultados provienen del fichero `testcases.txt` (salida del simulador) y resumen las métricas promedio para cada combinación algoritmo $\times$ workload.

Tabla Resumen General

Algoritmo	Workload	Avg Turnaround	Avg Response	Throughput
FCFS	I/O	2517.54	265.98	0.034
	Bound			
	(90/10)			
FCFS	Balanced	3746.24	277.57	0.021
	(50/50)			
FCFS	CPU	5606.55	324.30	0.015
	Bound			
	(10/90)			
Round Robin (q=5)	I/O	2403.41	195.69	0.033
	Bound			
	(90/10)			
Round Robin (q=5)	Balanced	3552.23	203.33	0.021
	(50/50)			
Round Robin (q=5)	CPU	5421.26	215.63	0.016
	Bound			
	(10/90)			
SJF	I/O	1559.74	773.73	0.034
	Bound			
	(90/10)			
SJF	Balanced	2229.49	1099.37	0.021
	(50/50)			
SJF	CPU	3859.01	1012.13	0.016
	Bound			
	(10/90)			

*Nota: Los valores en **negrita** representan los mejores resultados para cada métrica por workload.*

Comparación por Métrica

Turnaround Time (menor es mejor)

Workload	FCFS	Round Robin	SJF	Ganador
I/O Bound (90/10)	2517.54	2403.41	1559.74	SJF (-38% vs FCFS)
Balanced (50/50)	3746.24	3552.23	2229.49	SJF (-40% vs FCFS)
CPU Bound (10/90)	5606.55	5421.26	3859.01	SJF (-31% vs FCFS)

Conclusión: SJF reduce el turnaround promedio entre 31-40% comparado con FCFS.

Response Time (menor es mejor)

Workload	FCFS	Round Robin	SJF	Ganador
I/O Bound (90/10)	265.98	195.69	773.73	Round Robin (-26% vs FCFS)
Balanced (50/50)	277.57	203.33	1099.37	Round Robin (-27% vs FCFS)
CPU Bound (10/90)	324.30	215.63	1012.13	Round Robin (-34% vs FCFS)

Conclusión: Round Robin mejora el tiempo de respuesta entre 26-34% comparado con FCFS. SJF tiene el peor response time (294-441% mayor que RR).

Throughput (mayor es mejor)

Workload	FCFS	Round Robin	SJF	Observación
I/O Bound (90/10)	0.034	0.033	0.034	Empate técnico
Balanced (50/50)	0.021	0.021	0.021	Idéntico
CPU Bound (10/90)	0.015	0.016	0.016	RR/SJF ligeramente mejor

Workload	FCFS	Round Robin	SJF	Observación
----------	------	-------------	-----	-------------

Conclusión: El throughput es prácticamente idéntico entre algoritmos para cada workload, dominado por la carga total de trabajo.

Análisis por Workload

Workload I/O Bound (90% I/O, 10% CPU)

Métrica	FCFS	Round Robin	SJF	Diferencia Max
Avg Turnaround	2517.54	2403.41 (-4.5%)	1559.74 (- 38%)	957.80
Avg Response	265.98	195.69 (-26%)	773.73 (+191%)	578.04
Throughput	0.034	0.033	0.034	~0

Mejor para Turnaround: SJF

Mejor para Response: Round Robin

Mejor para Throughput: Empate (FCFS/SJF)

Workload Balanced (50% I/O, 50% CPU)

Métrica	FCFS	Round Robin	SJF	Diferencia Max
Avg Turnaround	3746.24	3552.23 (-5.2%)	2229.49 (- 40%)	1516.75
Avg Response	277.57	203.33 (-27%)	1099.37 (+296%)	896.04
Throughput	0.021	0.021	0.021	0

Mejor para Turnaround: SJF

Mejor para Response: Round Robin

Mejor para Throughput: Empate

Workload CPU Bound (10% I/O, 90% CPU)

Métrica	FCFS	Round Robin	SJF	Diferencia Max
Avg Turnaround	5606.55	5421.26 (-3.3%)	3859.01 (-31%)	1747.54
Avg Response	324.30	215.63 (-34%)	1012.13 (+212%)	796.50
Throughput	0.015	0.016 (+6.7%)	0.016 (+6.7%)	0.001

Mejor para Turnaround: SJF

Mejor para Response: Round Robin

Mejor para Throughput: Round Robin/SJF

Gráfico Visual de Rendimiento Relativo

Turnaround Time (normalizado a FCFS = 100%)

Workload	FCFS	Round Robin	SJF
I/O Bound	100%	95%	62% (down)
Balanced	100%	95%	59% (down)
CPU Bound	100%	97%	69% (down)

Response Time (normalizado a FCFS = 100%)

Workload	FCFS	Round Robin	SJF
I/O Bound	100%	74% (down)	291% (up)
Balanced	100%	73% (down)	396% (up)
CPU Bound	100%	66% (down)	312% (up)

Análisis y discusión

A partir de los resultados medidos:

1. Throughput

- El throughput es similar entre algoritmos para cada tipo de workload (valores muy cercanos por escenario). Esto indica que la cantidad de procesos completados por unidad de tiempo estuvo dominada por la

carga total de CPU/I/O y la duración agregada de los bursts más que por la política de scheduling en estos experimentos.

2. Turnaround promedio

- SJF muestra consistentemente el menor Avg Turnaround en los tres escenarios, particularmente en el caso I/O-bound (1559.74 vs 2517.54 en FCFS y 2403.41 en RR). Esto es consistente con la propiedad de SJF de priorizar trabajos cortos, reduciendo el tiempo promedio de finalización.
- FCFS presenta el peor Avg Turnaround para todos los escenarios, especialmente en CPU-heavy (5606.55), por su naturaleza no-preemptiva y orden FIFO que penaliza procesos largos al inicio.

3. Response time promedio

- Round Robin con quantum = 5 produce los mejores (menores) Avg Response en todos los escenarios comparado con FCFS y SJF (ej. RR I/O-bound: 195.69 vs FCFS 265.98 y SJF 773.73). RR mejora la latencia inicial al repartir la CPU frecuentemente entre procesos.
- SJF tiene un Avg Response alto, especialmente en Balanced y CPU-heavy, porque prioriza procesos con ráfagas cortas; procesos largos pueden esperar mucho antes de obtener CPU y así su tiempo hasta el primer inicio aumenta.

4. Efecto del workload

- En general, a medida que el workload se vuelve más CPU-bound, avg turnaround y response tienden a incrementarse para FCFS y SJF, aunque SJF reduce el impacto relativo gracias a su preferencia por trabajos cortos.
- RR mejora el response en comparación con FCFS y SJF, especialmente para I/O-bound y workloads balanceados.

Recomendaciones

- Para minimizar turnaround promedio en un entorno mixto o I/O-dominante, SJF suele ser la mejor opción.
- Si la prioridad es reducir la latencia de respuesta (por ejemplo sistemas interactivos), Round Robin con quantum pequeño (ej. 4–8) es preferible.
- FCFS es simple pero puede tener un rendimiento pobre para cargas que incluyen procesos largos (CPU-bound).

Limitaciones y trabajo futuro

- La generación de bursts es aleatoria; para estudios estadísticos robustos conviene ejecutar cada combinación algoritmo×workload múltiples veces (p. ej. 30 replicaciones) y reportar medias y desviaciones estándar.
- Se podría exportar la traza y los datos a CSV para análisis en Python/R y generar gráficos (CDF, boxplots) de las métricas.
- Evaluar más políticas (Priority, Multilevel Feedback Queue) y medir efectos de diferentes quantums en RR.

- Añadir liberación de memoria y opciones de configuración (número de procesos, bursts por proceso, distribución de duraciones) vía línea de comandos o archivo de configuración.

Conclusión

Los resultados muestran las típicas ventajas y desventajas de cada algoritmo: SJF optimiza turnaround, RR reduce tiempo de respuesta, y FCFS es el menos eficiente en presencia de procesos largos. El throughput fue similar entre algoritmos en estas instancias porque depende más del total de trabajo y de la mezcla I/O/CPU que de la política en sí.

Uso de LLMs (documentación mínima)

Se consultó un LLM para:

- **Estructuras de datos:** Implementación de una cola de prioridad (priority queue) para el algoritmo SJF y cola FIFO para FCFS/RR en C.
- **Lógica de scheduling:** Asistencia en la lógica de inserción y extracción de procesos de las colas, manejo de estados y transiciones entre READY, RUNNING, WAITING.
- **Manejo de memoria:** Recomendaciones sobre gestión de memoria dinámica para las estructuras de colas y procesos.
- **Arquitectura del simulador:** Sugerencias sobre la organización del ciclo principal de simulación (tick-based) y actualización de estados.

Criterios finales de diseño: Las estructuras de colas, los algoritmos de inserción/extracción y la máquina de estados de procesos fueron ajustados por el equipo para garantizar: - Correcta priorización en SJF (menor burst primero) - Manejo apropiado de preempción en Round Robin - Transiciones de estado consistentes - Liberación completa de memoria al finalizar

Conversacion Completa para la implementacion: <https://chatgpt.com/share/6907b2cb-03f8-8006-b5ac-d120824325fc>